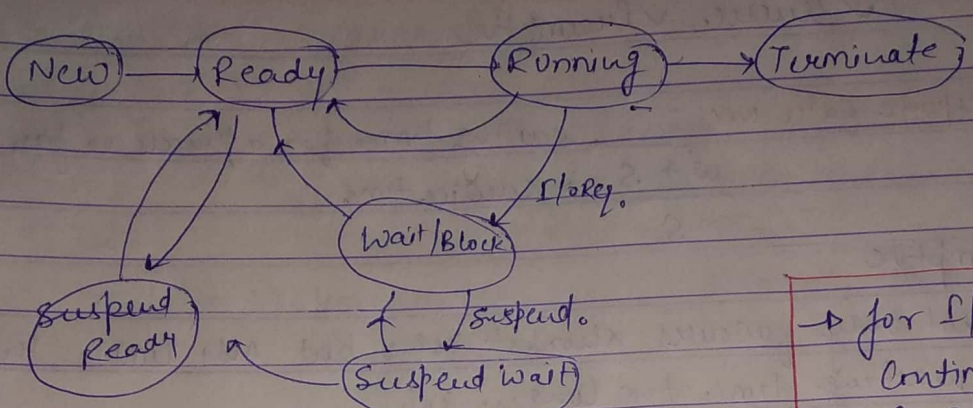
Process Management → Previous Year : Que 10,56.



→ for I/O process can continue, even if it is in SM.

→ long term scheduler [New → Ready]
→ Mid term [suspend Ready to New] (swapping)
→ short term [Ready to Run].
↓ Dispatcher.

✓ degree of multiprogramming – # jobs in Ready queue
✓ Throughput – # Jobs completed per unit time.
✓ Efficiency – Useful Time / Total time.

CPU scheduling :

① FCFS : ✓ Convoy effect (Starvation), ✗ Non-preemptive
✓ Queue, ✓ O(n)

② SJF : ✓ Non-preemptive, ✓ Convoy effect ✓ Min-Heap ✓ O(nlogn)
Process ✓ delete/insert

SRF (Preemptive) ✓ It give min – Avg waiting & Turn around time

Predictive Technique → Static [Process size, Process type etc].
→ Dynamic $\left[T_{n+1} = \alpha t_n + (1-\alpha) T_n\right]$.
↓
Exponential Avg / Aging

Scanned by CamScanner

fork() : $2^n - 1$

(3) Round Robin: ✓ Queue ✓ Preemptive

(4) Highest Response Ratio Next: $\frac{w + s}{s}$ → w = waiting time for a process so far, s → service time

↓

Non-preemptive

** → It not only favours shortest jobs but also limits the waiting time for longer job.

(5) Priority:
- static : Non-preemptive
- Dynamic : preemptive.

(7) Multiqueue : Can cause starvation

(8) Multiqueue feedback : No starvation

✓ Dispatch Latency : time taken to stop one process & start another process.

✓ Scheduling Length : $\max^m CT - \min^m AT$.
(Completion time) − (Arrival time)

CT : time taken to complete [start from AT] → By default 0!
Turn Around : CT − AT
Waiting Time : Turn Around time − Burst Time

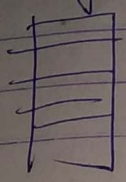
Scanned by CamScanner

→ GANG scheduling

Threads:

→ Traditional multiprogramming:

→ Using fork().



Each process has PCB [doubly LL used], which has own
→ stack, → Register → Data → code etc.
↓
Repetitive memory

↳ System call Require Context switching.

Thread:

(Virtual memory, Heap & Address space, Global Variable)

Share: File Descriptor, Code, Data/heap,

Not share: stack, Register, PC

↳ No system call [for user Level thread].

User Level	Kernel Level
→ System call not require.	→ System call requires but Less expensive Compared to traditional.
→ No Context switching. [or Very fast Context switching]	→ Unfair scheduling resolved.
→ Blocked system call [if service request is not available]. So, it will block the whole task if one task is Blocked [All other Also blocked]	→ Slow/expensive Context switching.
→ Unfair scheduling [OS not know about how many User Level threads with 1 KT].	→ One thread will not block other threads.

↳ kill all pro

Scanned by CamScanner

→ Not scheduled by the kernel.

Sp.: ~~In both Asynchronous I/O and~~ In synchronous I/O an ISR (Interrupt Service Routine) is invoked after completion but not in Asynchronous.

Scanned by CamScanner

Deadlock

≠ Necessary Condition
- Mutual Exclusion
- Hold & Wait
- Non-preemptive
- Circular wait

Que:

Process : P_1 P_2 — — P_n

Req. Req : x_1 x_2 — — x_n

Assigned : (x_1-1) (x_2-1) — — (x_n-1) → Max^m No of Resources that will cause deadlock.

$+1$ → Min^m no of resources that will not cause any deadlock.

→ Deadlock Prevention:
- → Mutual Exclusion : Allowed shared Resources if possible.
- → Hold & Wait : Allow All resources before execution.
- → Non-preemptive : Allow preemption w.r.t to Resources.
- → Circular wait : Resources can be allocated either in Increasing or Decreasing order.

Only this is possible practically

→ Deadlock Avoidance:
- → Banker's algorithm [Safe Algo].

→ Deadlock detection & Recovery
- → Single Instance [cycle present]
- → Multiple Instance [Safety Algo]
- → Resource
  - → Preemption
  - → Rollback
- → Process
  - → kill 1 Process
  - → kill all process

Scanned by CamScanner

Ostrich Algorithm:

Just ignore the deadlock. We act there is no deadlock. Eg. In Window & Linux :- Restart the system.

Scanned by CamScanner

Memory Management

(Header, Text segment, Data segment, Relocation info, Symbol table, debugging info)

Text file → Compiler → Assembly code → Assembler → Object Code (In Hard Disk)

Linker

- Link all object codes of shared lib.
- fill symbol table [one to another]
- Relocatable to another Relocatable Address (Relocation of Address)
- Symbol Resolution [for shared library].
- Linker → Dynamic linking (stub or glue).
  → Static linking
  (Related to shared library available)

Loader

- Programming loading Hard Disk to MM.
- Relocation to Absolute code.
  (Using symbol / object code info)

Binding:

1. Compile time binding: Absolute address is known for process in RAM.
2. load time Binding: Address relocation to absolute address.
3. Execution time: At Run time, Address might change. [Uses, Compaction etc.]
4. Linker time Binding:

Scanned by CamScanner

Imp: ⊢▷DL can cause security concerns, bcoz path for searching lib not known till run time.

Note: Dynamic linking may cause more page fault.

Current scenario:

Compile time: Symbolic names → Relocatable Address

Link time: Relocatable Add. → Relocatable Add.

Load time: Relocatable Add. → Absolute Address

Run time: Dynamic libraries → OS support

2-Pass Assembler:

1st Pass: allocate space for literals.

2nd Pass: object Program is generated.

✓ Note: Hardware support for MM is required if we are using Virtual memory. If not then not required.

└▷ Not Always true. Confusion.

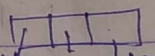
↓

Bcoz for Protection, it might required.

Scanned by CamScanner

Memory Allocation Technique :

① Continuous → fixed partitioning
- ↳ fixed process size
- ↳ Internal fragmentation
- ↳ External fragment.
- ↳ limited Multiprogn. degree

→ Dynamic partitioning
- ↳ No Internal fragmentation
- ↳ External fragme.
- ↳ Bit map & Linked list [ full/hole start end]
  - for single unit, 1 bit
  - [0 - unoccupied]
  - [1 - occupied]
- ↳ Compaction used

② Variable partitioning → first fit, Best fit, Next fit, worst fit
- ↳ Practically best.
- ↳ Practically not Good.

Quick fit
- ↳ Maintain a LL for most frequently Used process size.

Sup. Overlays : → Load a part of process
- → done by user not by OS. [virtual memory is done by OS].
- ↳ used in Continuous allocation.

Scanned by CamScanner

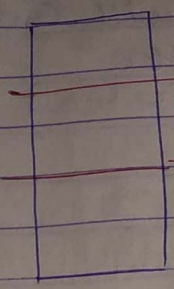
Note: If word size is given then convert all Address to word otherwise Assume 1 word = 1 byte

Memory Allocation

- Continuous
  - Static
    - Fixed (IFT, EFT)
    - Variable (EFT, EFT)
  - Dynamic
    - IFX
    - EFT ↑ [Compactation]
- Non-Continuous
  - Paging
    - Load time binding not possible.
    - EFX
    - H/W support required for Paging, either we use VM or not.
    - Internal fragmentation present.
  - Segmentation
    - Use: Let f() can be divided into two Pages. [Sometimes, a module of code get divided into diff Pages] ⇒ Use segmentation
    - close to user perceptive.
    - segment table is Maintained & will be present in MM.
    - segment pointer is used to store base/start address for ST.

Seg

Segmentation:



Segment table

S.No.	Base	Limit

Logical Address

S.No.	offset

↓ Segment No.

offset < limit

↓ Yes → Base + offset

↓ No (Invalid Add.)

Que.

Segment	Base	Length
0	330	124
1	876	211

Logical Address

0	99

⇒ Physical Address

99 < 124

↓ Yes

99 + 330 | Ans.

Segmentation with Paging:

(Video 76)

↳ Program is divided into the segments. [S_1, S_2, S_3 ...]

↳ MM is divided into the frames. [f_1, f_2 ... f_n]

↳ Each segment is divided into pages. [S_1(Pages), S_2(Pages) ...]

↳ Each segment has own Page table. [PT_1, PT_2 ...]

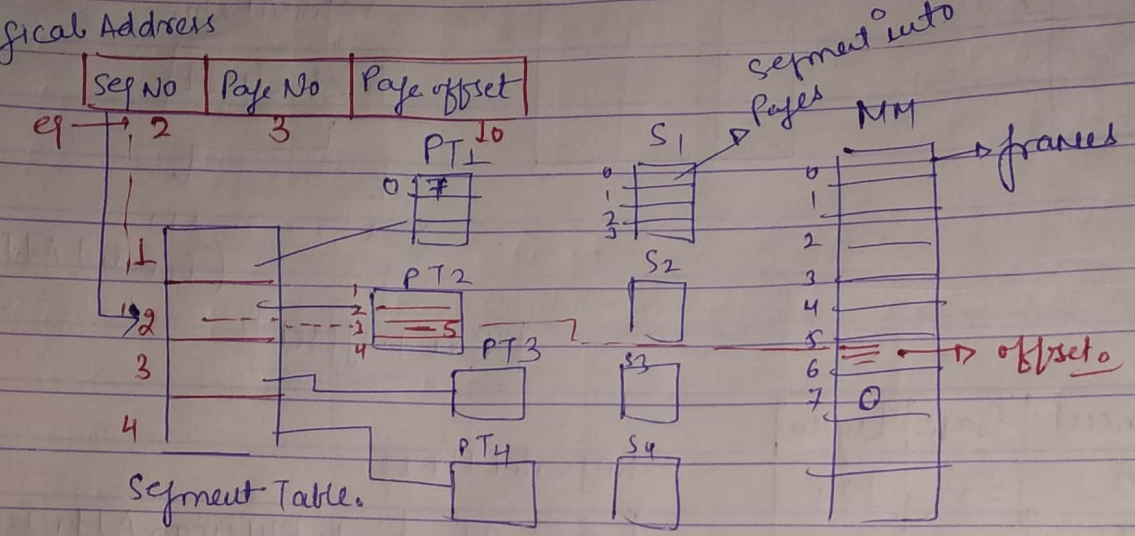
Exp: ↳ Each page table should fit into a single page. We need one more table to maintain info of Page tables [Correspond to all segments]

⇒ Segment Table

Scanned by CamScanner

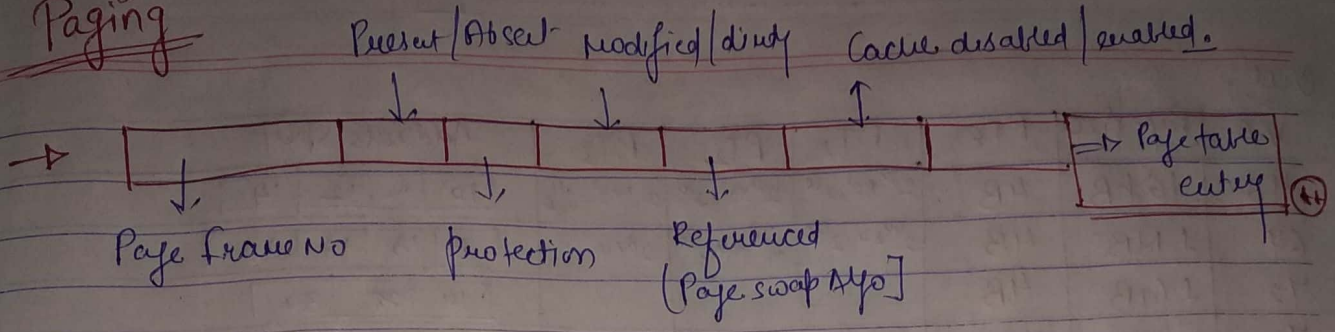
Note: ft mmd.

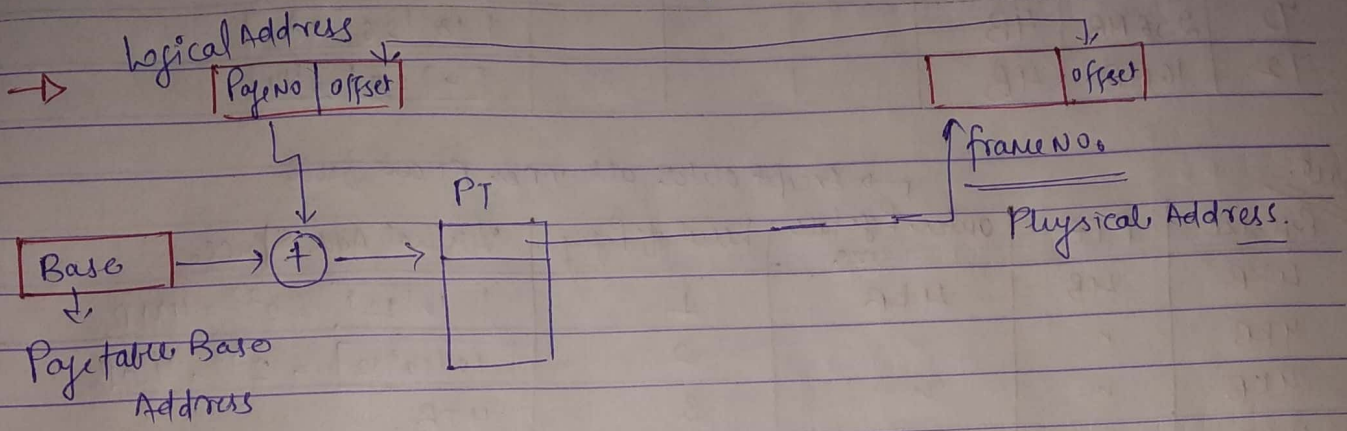
→ segment table should also present in frames.
So, CPU will have one register that will contain the info of base of segment table.



Scanned by CamScanner

Paging





Multi Level Paging:

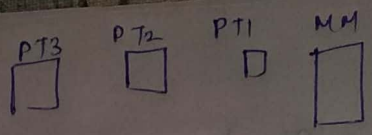
[Video 33, 34]

Let Page size / frame size = 4KB

and Page table size is = 6KB.

=> PT can't fit into the single frame / Page. So, multi-Level paging required.

Scanned by CamScanner



Que:

VA	Page size	PTE	PT_1	PT_2	PT_3	Address split
48	16 KB	4B	$2^{34} \times 2^2 = 2^{36}$	$2^{22} \times 2^2 = 2^{24}$	$2^{10} \times 2^2 = 2^{12}$ B	10 | 12 | 12 | 14
64	1 MB	4B	$2^{44} \div 4 = 2^{46}$	$2^{26} \div 4 = 2^{28}$	$2^6 \div 4$	8 | 18 | 18 | 20
72	1 GB	4B				14 | 28 | 30
72	256 MB	4B				18 | 26 | 28
72	16 MB	4B				4 | 24 | 24 | 24

Que

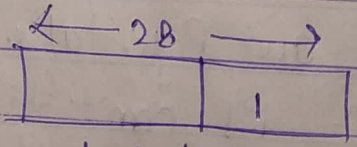
→ except outer, all inner PT are full

Page size	PTE	Outer Page table size	Level of Paging	Virtual Add. Space
4 KB	4B	4 KB	1	10 | 12 $= 2^{22} = 4$ MB
4 KB	"	"	2	10 | 10 | 12 $= 2^{32} = 4$ GB
4 KB	"	"	3	4 TB.
4 KB	"	256 B	1	6 | 12 $= 2^{18} = 256$ KB
4 KB	"	"	2	256 MB
4 KB	"	"	3	256 GB.

Que.

VAS	PS	PTE
4 MB	4 KB	$\leq$ 4B
4 GB	128 KB	$\leq$ 4B
128 TB	32 MB	$\leq$ 8B
256 MB	$\geq$ 32 KB	4B ⟹ Soln.
512 KB	$\geq$ 1 KB	2B
16 KB	$\geq$ 256 KB	4B

→ Such that it should fit in 1-level page table.



Let page size = P.

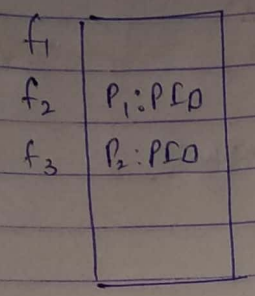
$$P \geq \frac{2^{28}}{P} * 4$$

$$\Rightarrow P^2 \geq 2^{30}$$

$$\Rightarrow \boxed{P \geq 2^{15}}$$ Ans.

Scanned by CamScanner

Inverted Page table:



→ Single page table for all process.
→ frame no is used as indices.
→ Entry contain Page no with PID
→ Space space, searching time Consuming.

Imp overhead in Paging:

$$n * \left(\frac{P}{2}\right) + \left(\frac{S}{P}\right) * e$$

P: page size
n: ~~Level~~ # process divided into 'n' sections
S: Avg Virtual Add space
e: entry.

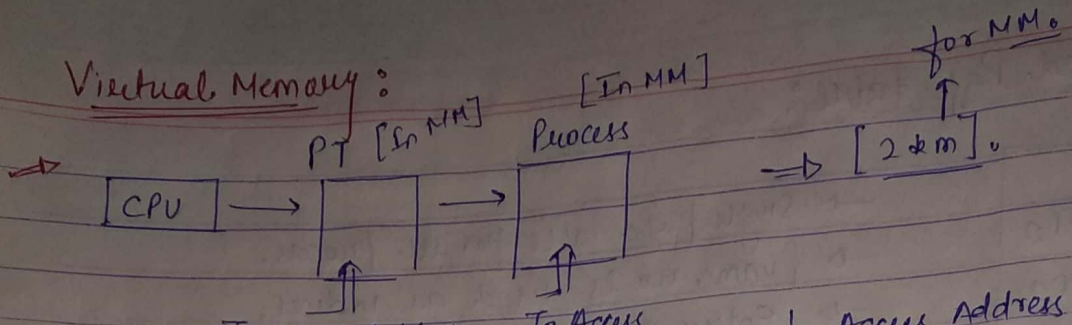
optimal page size ⟹ $P = \sqrt{2es}$ ← memory wasted.

[when n=1].

$$P = \sqrt{\frac{2es}{n}}$$

Scanned by CamScanner

Virtual Memory :



⇒ Same for K-Level Paging ⇒ [k * m + m]

to Access Address.

⇒ With TLB:

$$p(t+m) + (1-p)(t + k*m + m)$$

- p: Hit Ratio
- t: TLB time
- k: Level of paging

⇒ Avg. time to Run a Instⁿ:

Virtual to Physical Add

(VA → PA) + [fetch the word from MM for process].

$$P_t[t + P_c(c) + (1-P_c)*M] + (1-P_t)[t$$

(to ~~Access Memory~~)

$$t + (1-p)[k*m] + c + (1-P_c)(m)$$

- $t + (1-p)[k*m]$: to Convert VA → PA.
- $c + (1-P_c)(m)$: to Access Memory word

Scanned by CamScanner

Stack Based Algo : Page(B) ⊆ Page(4)
Subset.

VM
- Frame Allocation
  - Equal allocation
  - Weighted allocation
  - Dynamic (priorities Based)
- Page Replacement
  - Local [LRU, optimal, fifo, LRU]
  - Global

Belady [Not stack based]

Only data structure differ

⇓

Not practically used
[one process with highest priority will take all the frames].

✓ Working set:

- Not page replacement Algo
- Used for locality of references.
- Window size used.

Eg. WS = 4.

1	2	3	1	2	4	1	4	2	1	5	1	2	4	2	1

At end. {1 2 3 1} ⇒ WS = {1, 2, 3}. ++.

- Used for dynamic allocation. Even though 4 frames are allocated but most of the time it is using '3'.

Demand is increasing or decreasing.

⇓

So, we can allocate free frames to other process whose requirement is high. ++

✓ Avg. Working set = $\frac{\text{Sum of all set [with window size]}}{\#\text{References.}}$

Scanned by CamScanner

Initial WS given

Que a d e | c c d b c e c e a d

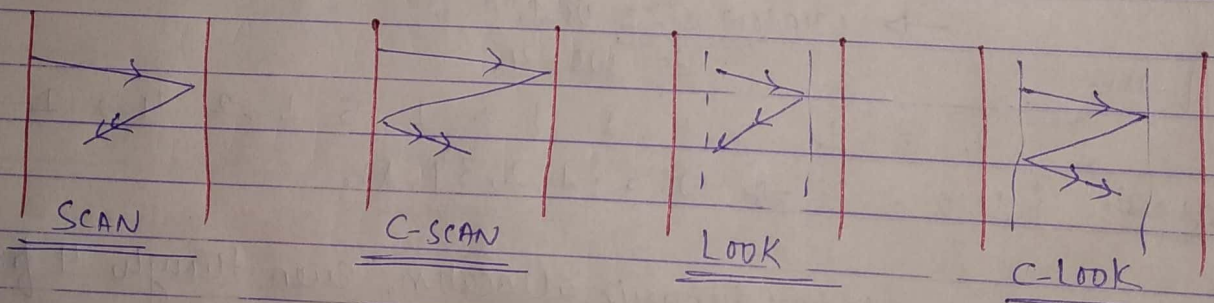
$\{a,d,e\} \Rightarrow \{a,d,e,c\}^{4} \Rightarrow \{a,e,c\}^{3} \Rightarrow \{e,c,d\}^{3} \Rightarrow \{c,d,b\}^{3}$

$|WS|=4$

$\Rightarrow \{c,d,b\}^{3} \Rightarrow$

Avg $|W^3| \Rightarrow \frac{4+3+3+3+3+--}{10}$

✓ File Disk scan:



Scanned by CamScanner